NLP HW3 — Q2: WhoDunnit (Multi-Agent Systems)

Gal Barak

July 2026

2.1 Naive Agent Implementation (Analysis)

We ran the naive baseline with the default configuration and no additional flags (`python run_game.py --iterations 3`: 3 suspects, `max_turn_count = 11`, no summarizer, no internal thought). *Alex* is the Accuser and *Beth* the Intel agent.

What specific behavioral problems did you identify in the initial run of the naive agents? Provide at least two concrete examples from your `game_log`.

Problem A — Analysis paralysis: the Accuser will not accuse even when the culprit is already uniquely determined.

- **Game 1** (culprit = Suspect 3): only Suspect 3 wears square glasses, so Beth's answer `square eye glasses` → `Suspect 3` already fixed the culprit. Alex still asked four more broad questions and never accused, hitting the turn limit.
- **Game 2** (culprit = Suspect 3): `blue eye color` → `S2, S3` and `circular eye glasses` → `S1, S3` intersect to {S3}, yet Alex kept questioning until timeout.

Problem B — No cross-turn reasoning. The Accuser treats each answer in isolation and never intersects the narrowing suspect lists into a running candidate set, so it cannot recognize that only one suspect remains — the root cause of Problem A.

Do the agents dynamically utilize the full spectrum of their available action space, or do they prematurely converge on a single, repetitive action type? Analyze how this behavioral pattern affects their overall progress.

They **converge**. Alex issues only `request-broad` in all three games (zero `request-specific`; `accuse` only once, in Game 3), which in turn forces Beth into `respond-broad` only. Because the terminal `accuse` action is under-used, information accumulates but is rarely acted upon, so games stall: Games 1 and 2 exhaust all 11 turns without an accusation, ending in timeout rather than a decision.

Does the Accuser make a blind accusation prematurely, or does it get stuck in an endless loop until the `max-turn-count` is reached? Why do you think it struggles to finalize the decision?

It gets stuck in an **endless questioning loop**, not a premature accusation: Games 1 and 2 reach `max-turn-count` with no accusation, and Game 3 accuses correctly only after redundant confirmation. It struggles to finalize because:

1. **Stateless prompting** — no maintained candidate set, so it never sees that one suspect remains;
2. the naive prompt **forbids reasoning**, so the model defaults to asking one more question rather than committing;

3. **no firm stopping criterion**, so "accuse when certain" never triggers.

2.2 Context Management via Summarization

We implemented `SummerizerAgent.summarize` as an evidence-preserving LLM rewrite of the channel (`temperature`=0.2, prompt instructed to keep every per-suspect confirmed/excluded attribute and never invent facts) and ran with the summarizer enabled (`python run_game.py --include-summerizer --iterations 3`). The Summarizer is *Charlie*.

Did the summarizer inadvertently remove any critical clues that led to the Accuser failing to identify the culprit? Provide a specific example from the logs.

The decisive clues were generally *kept*, but the summarizer distorted evidentiary *certainty*. In Game 1 Beth stated definitely that `circular eye glasses` → `Suspect 1, Suspect 2, Suspect 3`, yet Charlie rewrote this as "Suspect 1: *possibly* has circular eye glasses" (and likewise for S2, S3), downgrading a confirmed fact to an uncertain one. This ambiguity plausibly caused Alex to redundantly re-ask "circular eye glasses." Alex *also* re-asked "blue eye color," but that clue was *not* downgraded (the summary Alex saw still read "Suspect 3: has blue eye color"), so this second repeat is better explained by the loss of the verbatim Q&A record — the per-suspect summary dropped Alex's memory of which questions it had already asked. Either way Alex exhausted its turns without accusing — even though the culprit-identifying clue (only Suspect 3 has blue eyes) had actually been preserved.

Did the reduction in context length result in more coherent questions responses from the Accuser and Intel agents, or did the loss of "raw" dialogue history make their behavior less natural?

Mixed, but coherence improved in 2 of 3 games. The per-suspect organized summaries (e.g. Game 2: "Suspect 2: blue eye color, wears square eye glasses, has long hair") kept the Accuser tightly focused on the correct suspect and pushed it toward targeted `request-specific` questions — an improvement over the naive baseline's endless broad queries. But when a summary was vague (Game 1's "possibly has"), the loss of the verbatim Q&A record removed Alex's memory of what it had already asked and it repeated questions. So compression helps when the summary is faithful and structured, and hurts when it introduces ambiguity.

Based on your observations, do you believe this summarization approach would scale to a game with a much larger suspect pool (e.g., `suspect-count=20`)? Explain your reasoning.

Not without changes. (1) A free-text LLM rewrite is lossy: the "possibly" distortion seen at 3 suspects will compound at 20, and a single weakened or dropped fact can leave the culprit ambiguous. (2) The summary must record facts for many suspects, so its length grows with the pool and the "concise channel" benefit erodes. (3) The real bottleneck is the Accuser's reasoning and stopping ability, which summarization does not address — and a third of all turns are consumed by summarizing. A deterministic, structured state (a maintained per-suspect table of confirmed/excluded attributes) would scale far better than a natural-language summary.

2.3 Implementing Reasoning Agents

We implemented `take_turn_thought` for both the Intel and the Accuser (`python run_game.py --allow-internal-thought`). Each agent may now reason freely, but still emits the environment's required final `ACTION/INFO` block.

Detail your architectural decisions regarding prompt engineering. What specific intermediate reasoning steps did you introduce to guide the agents, and what did you want each agent to analyze internally before generating its environment-compliant response?

To keep chain-of-thought compatible with the environment's schema, we parse only the *last* `ACTION/INFO` block (helper `_parse_final_action`: the `ACTION` keyword is read from its first line, while the `INFO` field keeps every consecutive suspect-listing line so a broad answer wrapped over several lines is not truncated), so reasoning never leaks into the action, and we regex-normalize any accusation to `"Suspect N"` (falling back to another question when no suspect number is given) so `Env.test_accusation` cannot crash on verbose output.

- **Intel** — internal steps: (1) classify the question as *specific* (Yes/No) or *broad*; (2) identify the relevant attribute; (3) check each suspect against it. Goal: verify per-suspect facts before answering, so broad lists are complete and Yes/No answers are correct.
- **Accuser** — internal steps: (1) extract the culprit's attributes from its description; (2) for every suspect, mark which culprit attributes are confirmed/excluded and maintain the set of suspects consistent with *all* evidence so far; (3) accuse iff exactly one candidate remains, otherwise ask the single most-informative (preferably broad) non-repeated question. The intended internal artifact is the *running candidate intersection* — exactly what the naive agent never computed. We also pass `turns_left` so it trades certainty against the clock.

Evaluate whether this "Chain-of-Thought" mechanism successfully mitigated the Task 1 failure modes. Did adding this internal reasoning step improve agent coordination, action-space utilization, or overall success rate compared to the naive baseline?

Yes, clearly, on all three axes:

- **Action-space utilization:** the Accuser now actually reaches `accuse` and mixes `request-broad`/`request-specific`; the terminal action is no longer neglected. Intel likewise exercised both actions: `respond-broad` throughout the 3-suspect runs and `respond` (Yes/No) in the scaling runs (e.g. `Does Suspect 3 have blue eyes?`→ *Yes* at N=5, and `Does Suspect 8 have a bronze watch?`→ *Yes* at N=16).
- **Coordination:** Intel's lists are accurate and the Accuser *consumes* them by intersecting — e.g. Game 1 (`black hat`→{S3}, `square glasses`→{S3}) and Game 3 (`green eyes`→{S1,S2} ∩ `red shoes`→{S1,S3} = {S1}).
- **Success rate:** 2/3 solved in the 3-suspect batch (plus a standalone game also solved), versus ≈0–1/3 for the naive baseline. The endless-questioning loop is largely fixed.

If the agents still fail, discuss the underlying causes. If successful, scale the suspect count and determine the maximum your agents can process (subject to the turn limit and API rate restrictions); analyze the factors that contribute to failure at scale.

Residual failure. In one 3-suspect game (Game 2) the intersection was already {S3} after just *two* broad answers (`black hat`→{S1,S3} ∩ `brown eyes`→{S2,S3} = {S3}), yet the Accuser asked three more broad questions and then a redundant specific question, and timed out. This is a symptom of *over-verification* (see below): the model reaches the answer but keeps asking, and with only ≈6 Accuser moves at the 11-turn limit, one or two wasted questions cause a timeout.

Scaling. We increased the suspect count with the turn limit set to 3× suspects. All configurations were **solved correctly**:

- **5 suspects** (15 turns): accused Suspect 3 correctly.
- **8 suspects** (24 turns): accused Suspect 3 correctly. The intersection was already {S3} after *2* questions (`green eyes`→{3,7} ∩ `black hat`→{3,5,6}), but the Accuser asked *7* before accusing.
- **12 suspects** (36 turns): accused Suspect 1 correctly. The intersection collapsed to {S1} after *4* questions, yet the Accuser asked *10* before accusing.
- **16 suspects** (48 turns): accused Suspect 8 correctly. The intersection was {S8} after *5* questions, but the Accuser asked *8* (one of which showed chain-of-thought text leaking into the `INFO` field; the stray answer was ignored and the next turn accused correctly).
- **20 suspects** (60 turns): accused Suspect 16 correctly. The intersection collapsed to {S16} after *4* broad questions (`basketball` ∩ `black hat` ∩ `white shoes` ∩ `brown eyes`), but the Accuser asked *9* before accusing.

So the agents' reasoning reliably scales to **at least 20 suspects** (the largest we tested), given a turn limit of ≈ 3× the suspect count. A first attempt at N=20 launched on an already-spent daily budget had been cut off by the Groq free-tier daily token cap (`Limit 100000`) after only 7 answered questions; re-running on a fresh budget solved it cleanly — confirming that the cap bounds *throughput*, not the agents' ability.

Factors that limit scaling. (1) **Over-verification** is the dominant inefficiency: the Accuser consistently asks several more questions than needed to reach a singleton (needed vs. asked was 2 vs. 7 at N=8, 4 vs. 10 at N=12, 5 vs. 8 at N=16, 4 vs. 9 at N=20). This is harmless when turns are plentiful but is precisely what causes timeouts under a tight limit — so the required turn budget, not correctness, is the practical ceiling. (2) **API budget:** each 70B turn (≈1024-token thoughts plus an Intel prompt that grows with N) is token-expensive. A single N=20 game fits a fresh daily budget, but launched on an already-spent budget it hit the Groq free-tier **daily token cap** (`tokens per day: Limit 100000`) after just 7 questions, and on an earlier day even N=8/N=12 capped mid-game. So throughput — how many and how large the games can be per day — is bounded by the budget even when a single game's reasoning would succeed. (3) **Intel accuracy at scale:** with 20+ descriptions to scan per broad question there is more room for a missed or misclassified suspect, which would corrupt the intersection.

Propose three modifications that could further enhance agent performance and improve the overall success rate of the system.

1. **Deterministic candidate tracking in code.** Maintain a per-suspect table of confirmed/excluded attributes parsed from the Q&A and auto-accuse once a single candidate remains; let the LLM only *choose questions*. This removes reliance on the model's mental set-intersection and directly eliminates the over-verification that causes timeouts.
2. **Information-maximizing, de-duplicated questions.** Instruct the Accuser to pick the attribute whose answer most evenly splits the remaining candidates (binary-search/entropy style) and forbid repeating any prior question — fewer turns needed, so larger pools fit under the turn limit.
3. **Token-budget efficiency.** Cap reasoning length, use the small model for the narrowing questions and the 70B model only for the final accusation, and add a `time.sleep`/backoff-and-retry on 429 — so larger suspect pools fit within the daily and per-minute limits.